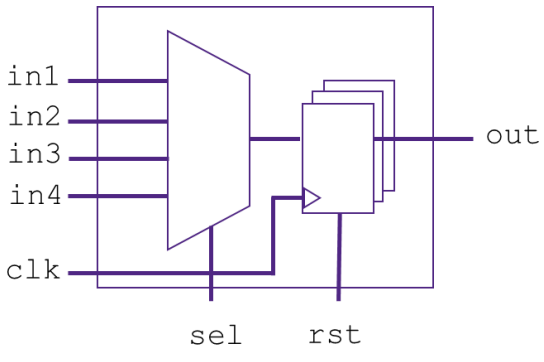
DC-LAB 0*: Getting familiarized with DC shell*

- Copy the lab directory (“FE”) to your personal directory and then go to the new directory:

```
UNIX% cd <your_personal_directory>
UNIX% cp -rf <parent_dir>/DCLAB/FE ./
UNIX% cd FE/LAB0
```

- We will be working with the **Parametrized 4-input MUX with output register** you worked on (Verilog Lab 1). It is instantiated within design **top**. Review the provided Verilog files (“gedit”, “vim” or “nano” are typical text editors):




```
UNIX% gedit rtl/top.v
UNIX% gedit rtl/mux4_registered.v
UNIX% gedit rtl/mux4.v
UNIX% gedit rtl/register_bank.v
```

  You can replace the “mux4_registered” within top with your own module.

- Review the run script “dc.tcl” (it contains the DC run commands for **technology configuration** and **RTL reading**):

```
UNIX% gedit scripts/dc.tcl
```

- Open a DC shell:

```
UNIX% dc_shell
```

- Within “dc_shell”, source the DC run script (**batch** execution). Once the script has finished running, the technology libraries will be set and the design will be loaded into DC memory (translated into a generic unmapped netlist).

```
dc_shell> source -echo -verbose scripts/dc.tcl
```

  ✓ **Tip:** Alternatively, source the script directly while opening the DC shell: `UNIX% dc_shell -f scripts/dc.tcl`

  ✓ **Tip:** Read the log printed by Design Compiler in the terminal after sourcing the file to understand what each command is doing. Each command is “echoed” and its result is displayed.

- Try out the command help to check available commands:

```
dc_shell> help report*
dc_shell> help get*
```

- Check the documentation (“man” for manual) of some commands:

```
dc_shell> man report_timing
dc_shell> man compile_ultra
```

- Try out some of the commands mentioned during the course. For example:

```
dc_shell> get_designs
dc_shell> get_cells
dc_shell> report_cell [get_cells -hier]
dc_shell> report_collection [get_cells] -columns {full_name ref_name is_sequential
area}
dc_shell> list_attributes -class cell -application
dc_shell> get_ports
dc_shell> all_inputs
dc_shell> get_libs
dc_shell> report_lib <lib_name>
dc_shell> get_lib_cells <library_name>/<lib_cell>
dc_shell> report_lib <lib_name> <lib_cell> -timing
dc_shell> get_attribute <lib_name> time_unit_name
dc_shell> report_cell
```

- Save the last report into an output file in the folder “reports”:

```
dc_shell> report_cell > reports/cells.rpt
```

- Open the file from the “dc_shell” by adding the command “sh” before:

```
dc_shell> sh gedit reports/cells.rpt
```

- Open the GUI (called “Design Vision”) and review the design:

  ```
  dc_shell> start_gui
  ```

  In the window “Logical Hierarchy” you can see the “top” design. Right-click gives you some options. Select “Schematic View” and with double-click over the shown symbol you can see the inside of the design.

  Play around in the GUI. Right-click on cells, see what information you can learn from them. Explore the toolbars and investigate what options Design Compiler offers.

- Write the generic unmapped netlist into a Verilog file:

  ```
  dc_shell> write_file -format verilog -hier -out unmapped.v
  ```

- Exit from the “dc_shell”:

  ```
  dc_shell> exit
  ```

- Check again the report created during the session but now directly from Linux shell:

  ```
  UNIX% gedit reports/cells.rpt
  ```

- Review the Verilog file you created for the **unmapped netlist (structural description)** and compare it to the **original RTL (behavioral description)**:

  ```
  UNIX% gedit unmapped.v
  ```